Technical Implementation Blueprint: Grand Theft Auto: South Coast (Build "QUAHOG")

The project architecture leverages a **decoupled, relational, data-driven approach**. This enables coding models to generate clean, modular scripts without losing context in a massive codebase.
By establishing a rigid data schema first, we ensure that gameplay systems (like property acquisition or dynamic weather handling) plug into a single, scalable source of truth.

1. Unified Game State & Player Wallet

To prevent duplicate states and currency bugs, the economy must rely on a strictly protected **Singleton Manager**.

```
// File: /Core/PlayerWallet.cs
using UnityEngine;
using System;

public class PlayerWallet : MonoBehaviour
{
    public static PlayerWallet Instance { get; private set; }

    [SerializeField] private int currentBalance = 0;
    [SerializeField] private int totalLifetimeEarned = 0;

    public static event Action<int> OnBalanceChanged;

    public int CurrentBalance => currentBalance;
    public int TotalLifetimeEarned => totalLifetimeEarned;

    private void Awake()
    {
        if (Instance != null && Instance != this)
        {
            Destroy(gameObject);
            return;
        }
        Instance = this;
        DontDestroyOnLoad(gameObject);
    }

    public bool AddFunds(int amount)
    {
        if (amount < 0) return false;

        currentBalance += amount;
        totalLifetimeEarned += amount;
```

```
        OnBalanceChanged?.Invoke(currentBalance);
        return true;
    }

    public bool DeductFunds(int amount)
    {
        if (amount < 0 || currentBalance < amount) return false;

        currentBalance -= amount;
        OnBalanceChanged?.Invoke(currentBalance);
        return true;
    }
}
```

2. Relational Asset Property Schema

Every buyable business (Asset) in the world map is treated as a modular data class, decoupled from its physical 3D world representation.

```
// File: /Systems/Assets/AssetProperty.cs
using System;

[Serializable]
public class AssetProperty
{
    public string propertyID;           // Unique Identifier (e.g.,
"PROP_NB_SCALLOP")[span_6](start_span)[span_6](end_span)[span_7](start
_span)[span_7](end_span)
    public string displayName;          // Display name (e.g.,
"Leviathan Scallop
Co.")[span_8](start_span)[span_8](end_span)[span_9](start_span)[span_9
](end_span)
    public int purchasePrice;           // Upfront capital
cost[span_10](start_span)[span_10](end_span)[span_11](start_span)[span
_11](end_span)
    public int baseDailyYield;          // Passive income per 24
in-game
hours[span_12](start_span)[span_12](end_span)[span_13](start_span)[spa
n_13](end_span)
    public bool isOwned;                // Ownership
state[span_14](start_span)[span_14](end_span)[span_15](start_span)[spa
n_15](end_span)
    public bool isDistressed;           // Operational friction flag
(stops yield if true)[span_16](start_span)[span_16](end_span)
    public int currentUpgradeLevel;     // Completed mission strands (0
to 3 multiplier)[span_17](start_span)[span_17](end_span)

    public int CalculateCurrentYield()
```

```
    {
        if (!isOwned || isDistressed) return 0; // Distressed assets
leak margin entirely[span_18](start_span)[span_18](end_span)

        // Progression multiplier: Level 0 = 100%, Level 3 = 250%
yield
        float multiplier = 1.0f + (currentUpgradeLevel * 0.5f);
        return UnityEngine.Mathf.RoundToInt(baseDailyYield *
multiplier);
    }
}
```

3. Yield Engine & Time Loop

The core progression engine listens directly to the global day/night cycle loop, calculating and dispensing yields at midnight.

```
// File: /Systems/Assets/RevenueManager.cs
using UnityEngine;
using System.Collections.Generic;

public class RevenueManager : MonoBehaviour
{
    [SerializeField] private List<AssetProperty> propertyDatabase =
new List<AssetProperty>();

    // In-game time tracking variables
    private float inGameHourTimer = 0f;
    private int currentHour = 0;
    [SerializeField] private float realSecondsPerInGameHour = 60f; //
1 Hour = 1 Real Minute

    private void Update()
    {
        TrackInGameTime();
    }

    private void TrackInGameTime()
    {
        inGameHourTimer += Time.deltaTime;
        if (inGameHourTimer >= realSecondsPerInGameHour)
        {
            inGameHourTimer = 0f;
            currentHour++;

            if (currentHour >= 24)
            {
                currentHour = 0;
```

```
                ProcessDailyYield(); // Trigger the midnight
drop[span_21](start_span)[span_21](end_span)
            }
        }
    }

    private void ProcessDailyYield()
    {
        int totalDailyPayout = 0;

        foreach (var property in propertyDatabase)
        {
            if (property.isOwned &&
!property.isDistressed)[span_22](start_span)[span_22](end_span)[span_2
3](start_span)[span_23](end_span)
            {
                int propertyPayout = property.CalculateCurrentYield();
                totalDailyPayout += propertyPayout;

                // Optional: Spawn a physical cash pickup at the
property's Transform position
here[span_24](start_span)[span_24](end_span)
                Debug.Log($"Generated ${propertyPayout} from
{property.displayName}");[span_25](start_span)[span_25](end_span)
            }
        }

        if (totalDailyPayout > 0)
        {
            PlayerWallet.Instance.AddFunds(totalDailyPayout);
            Debug.Log($"Total Daily Empire Yield Deposited:
${totalDailyPayout}");
        }
    }

    public AssetProperty GetPropertyByID(string id)
    {
        return propertyDatabase.Find(p => p.propertyID == id);
    }
}
```

4. Direct Acquisition Processor

This script processes transaction validations when a player interacts with a property's purchase marker.

```
// File: /Systems/Assets/AcquisitionEngine.cs
using UnityEngine;
```

```
public class AcquisitionEngine : MonoBehaviour
{
    [SerializeField] private string targetPropertyID;
    private RevenueManager revenueManager;

    private void Start()
    {
        revenueManager = FindObjectOfType<RevenueManager>();
    }

    public void AttemptPropertyPurchase()
    {
        AssetProperty targetProperty =
revenueManager.GetPropertyByID(targetPropertyID);

        if (targetProperty == null) return;
        if (targetProperty.isOwned) return; // Already
purchased[span_28](start_span)[span_28](end_span)

        int price = targetProperty.purchasePrice;

        // Transaction validation against the central Singleton
Wallet[span_29](start_span)[span_29](end_span)
        if (PlayerWallet.Instance.CurrentBalance >= price)
        {
            PlayerWallet.Instance.DeductFunds(price);
            targetProperty.isOwned =
true;[span_30](start_span)[span_30](end_span)[span_31](start_span)[spa
n_31](end_span)

            TriggerPurchaseFanfare(targetProperty);

UnlockPropertyMissions(targetProperty.propertyID);[span_32](start_span
)[span_32](end_span)
        }
        else
        {
            Debug.Log("Transaction Denied: Insufficient Capital.");
        }
    }

    private void TriggerPurchaseFanfare(AssetProperty property)
    {
        Debug.Log($"Empire Expanded! You now own:
{property.displayName}");[span_33](start_span)[span_33](end_span)
        // Insert audio fanfare trigger and UI overlay animation
initialization here
```

```
    }

    private void UnlockPropertyMissions(string id)
    {
        // Event broadcast to MissionManager to inject corresponding
asset mission flags[span_34](start_span)[span_34](end_span)
        Debug.Log($"Mission strings unlocked for asset ID:
{id}");[span_35](start_span)[span_35](end_span)
    }
}
```

5. Architectural Hierarchy Matrix

To maintain project hygiene while scaling, implement this directory structure across your version control repository:

```
Assets/
├── _Scripts/
│   ├── Core/
│   │   ├── GameManager.cs
│   │   └── PlayerWallet.cs
│   ├── Systems/
│   │   ├── Assets/
│   │   │   ├── AssetProperty.cs
│   │   │   ├── RevenueManager.cs
│   │   │   └── AcquisitionEngine.cs
│   │   └── Missions/
│   │       └── MissionManager.cs
│   └── World/
│       ├── TimeOfDayClock.cs
│       └──
WeatherStateController.cs[span_37](start_span)[span_37](end_span)
└── ScriptableObjects/
    └── Properties/
        ├── Prop_NB_Scallop.asset
        └── Prop_FR_TextileMill.asset
```

Next Step Verification

With this infrastructure verified, the game economy state machine can accurately track when a player purchases the *Leviathan Scallop Co.* or *Quequechan Mill #4*.

Setting up a robust, relational backend for a game's economy is a massive step forward. Moving beyond basic JSON files to a local SQLite database gives you a scalable, full-stack foundation to track complex empire states without data corruption.
Here is the architectural blueprint for both your persistence layer and the chop-shop weapon

integration.

1. SQLite Empire Persistence Layer

This script utilizes Mono.Data.Sqlite (standard in Unity) to handle local database connections. It ensures that the PlayerWallet and AssetProperty states are safely written to a local .db file and loaded when the game boots.

```
// File: /Core/Database/EmpireDatabaseManager.cs
using UnityEngine;
using System.Data;
using Mono.Data.Sqlite;
using System.Collections.Generic;
using System.IO;

public class EmpireDatabaseManager : MonoBehaviour
{
    public static EmpireDatabaseManager Instance { get; private set; }

    private string dbName = "SouthCoastEmpire.db";
    private string dbPath;

    private void Awake()
    {
        if (Instance != null && Instance != this) Destroy(gameObject);
        else Instance = this;

        // Sets the DB path to the persistent data directory (safe on
PC, Mac, and mobile)
        dbPath =
$"URI=file:{Path.Combine(Application.persistentDataPath, dbName)}";
        InitializeDatabase();
    }

    private void InitializeDatabase()
    {
        using (IDbConnection dbConnection = new
SqliteConnection(dbPath))
        {
            dbConnection.Open();

            // Create Player Wallet Table
            using (IDbCommand dbCmd = dbConnection.CreateCommand())
            {
                dbCmd.CommandText = @"
                    CREATE TABLE IF NOT EXISTS PlayerWallet (
                        ID INTEGER PRIMARY KEY AUTOINCREMENT,
                        CurrentBalance INTEGER NOT NULL,
```

```
                        TotalLifetimeEarned INTEGER NOT NULL
                    );";
                dbCmd.ExecuteNonQuery();
            }

            // Create Asset Properties Table
            using (IDbCommand dbCmd = dbConnection.CreateCommand())
            {
                dbCmd.CommandText = @"
                    CREATE TABLE IF NOT EXISTS AssetProperties (
                        PropertyID TEXT PRIMARY KEY,
                        IsOwned INTEGER NOT NULL,
                        IsDistressed INTEGER NOT NULL,
                        CurrentUpgradeLevel INTEGER NOT NULL
                    );";
                dbCmd.ExecuteNonQuery();
            }
            dbConnection.Close();
        }
        Debug.Log("[Database] SQLite Initialized.");
    }

    // --- WALLET PERSISTENCE ---
    public void SaveWalletState(int balance, int lifetimeEarned)
    {
        using (IDbConnection dbConnection = new
SqliteConnection(dbPath))
        {
            dbConnection.Open();
            using (IDbCommand dbCmd = dbConnection.CreateCommand())
            {
                // UPSERT logic: Insert or replace ID 1
                dbCmd.CommandText = $"INSERT OR REPLACE INTO
PlayerWallet (ID, CurrentBalance, TotalLifetimeEarned) VALUES (1,
{balance}, {lifetimeEarned})";
                dbCmd.ExecuteNonQuery();
            }
        }
    }

    // --- ASSET PERSISTENCE ---
    public void SavePropertyState(AssetProperty property)
    {
        using (IDbConnection dbConnection = new
SqliteConnection(dbPath))
        {
            dbConnection.Open();
            using (IDbCommand dbCmd = dbConnection.CreateCommand())
```

```
            {
                int owned = property.isOwned ? 1 : 0;
                int distressed = property.isDistressed ? 1 : 0;

                dbCmd.CommandText = $@"
                    INSERT OR REPLACE INTO AssetProperties
(PropertyID, IsOwned, IsDistressed, CurrentUpgradeLevel)
                    VALUES ('{property.propertyID}', {owned},
{distressed}, {property.currentUpgradeLevel})";
                dbCmd.ExecuteNonQuery();
            }
        }
    }
}
```

2. Quequechan Mill Chop-Shop & Arms Integration

Quequechan Mill #4 serves as both a chop shop and an illegal arms warehouse hidden within dead textile looms. To mechanize this, we create a scriptable object architecture that generates "weapon drops" or inventory unlocks based strictly on the player's ownership and upgrade level of that specific property.

The Weapon Data Profile

This defines what an individual weapon drop is.

```
// File: /ScriptableObjects/Weapons/WeaponProfile.cs
using UnityEngine;

[CreateAssetMenu(fileName = "NewWeaponProfile", menuName = "South
Coast/Weapon Profile")]
public class WeaponProfile : ScriptableObject
{
    public string weaponID;
    public string weaponName; // e.g., "Sawed-Off Pump", "Military
Surplus Rifle"
    public int baseDamage;
    public int requiredMillLevel; // The upgrade level the Mill must
be at to stock this
    public int marketValue;
    public GameObject weaponPrefab;
}
```

The Arms Inventory Manager

This manager cross-references the AssetProperty state of Quequechan Mill #4 to determine

which weapons the player can currently access or buy from the warehouse.

```
// File: /Systems/Assets/ChopShopArmsManager.cs
using UnityEngine;
using System.Collections.Generic;

public class ChopShopArmsManager : MonoBehaviour
{
    [Header("Mill Configuration")]
    [SerializeField] private string millPropertyID = "PROP_FR_MILL4";
// ID for Quequechan Mill #4
    private RevenueManager revenueManager;

    [Header("Warehouse Inventory Master List")]
    [SerializeField] private List<WeaponProfile> masterWeaponList;

    private void Start()
    {
        revenueManager = FindObjectOfType<RevenueManager>();
    }

    /// <summary>
    /// Returns a list of weapons currently available to the player
based on the Mill's operational state.
    /// </summary>
    public List<WeaponProfile> GetAvailableBlackMarketInventory()
    {
        List<WeaponProfile> availableInventory = new
List<WeaponProfile>();
        AssetProperty mill =
revenueManager.GetPropertyByID(millPropertyID);

        // If the player doesn't own it, or if it's currently
raided/distressed, the shop is closed.
        if (mill == null || !mill.isOwned || mill.isDistressed)
        {
            Debug.Log("[Arms Manager] The Mill is either unowned or
too hot right now. No inventory available.");
            return availableInventory; // Returns empty
        }

        // Filter master list by the Mill's current upgrade tier
        foreach (WeaponProfile weapon in masterWeaponList)
        {
            if (mill.currentUpgradeLevel >= weapon.requiredMillLevel)
            {
                availableInventory.Add(weapon);
            }
        }
```

```
        Debug.Log($"[Arms Manager] Mill Level
{mill.currentUpgradeLevel} active. {availableInventory.Count} weapons
available.");
        return availableInventory;
    }

    /// <summary>
    /// Simulates generating a physical crate drop at the mill
containing an unlocked weapon.
    /// </summary>
    public void GenerateWeaponDrop(WeaponProfile specificWeapon,
Transform dropLocation)
    {
        AssetProperty mill =
revenueManager.GetPropertyByID(millPropertyID);

        if (mill != null && mill.isOwned && !mill.isDistressed &&
mill.currentUpgradeLevel >= specificWeapon.requiredMillLevel)
        {
            Instantiate(specificWeapon.weaponPrefab,
dropLocation.position, dropLocation.rotation);
            Debug.Log($"[Arms Manager] Dropped crate containing:
{specificWeapon.weaponName}");
        }
    }
}
```